Politecnico di Milano

Ingegneria del Software – a.a. 2005/06

Appello del 24 luglio 2006

Cognome
Nome
Matricola
Sezione (segnarne una) Baresi ?, Ghezzi ?, Morzenti ?, SanPietro ?

Istruzioni

1. La mancata indicazione dei dati anagrafici e della sezione comporta l'annullamento del compito.
2. Al termine, consegnare **solo i fogli distribuiti** utilizzando il **retro** delle pagine in caso di necessità. Non separare questi fogli. Eventuali fogli di brutta, ecc. **non** verranno **in nessun caso** presi in considerazione. È possibile scrivere **in matita**.
3. È possibile consultare liberamente libri, manuali o appunti. È **proibito** l'uso di ogni dispositivo elettronico (quali calcolatrici tascabili, telefoni cellulari, ecc.).
4. Non è possibile lasciare l'aula conservando il tema della prova in corso.
5. Tempo a disposizione: 2h.
6. Punteggio totale a disposizione: 25/30. La sufficienza si raggiunge con 16 punti, a cui sommare il risultato delle prove di laboratorio.

Esercizio 1: .....

Esercizio 2: .....

Esercizio 3: .....

Totale .....

Esercizio 1 (punti 6)

L'ADT *Matrice* fornisce un'astrazione per una matrice quadrata di interi di ordine *n*. L'ADT presenta un metodo *leggiMatrice* che restituisce un vettore di interi che corrisponde alla matrice letta colonna per colonna. Il metodo legge il contenuto della matrice quadrata e riempie il vettore partendo dalla prima colonna. Ad esempio, se la matrice avesse dimensione 3, il vettore restituito dovrebbe contenere gli elementi a[0][0], a[1][0], a[2][0], a[0][1], a[1][1], a[2][1], a[0][2], a[1][2] e a[2][2]. Si supponga che l'ADT fornisca anche i metodi *lunghezza* ed *elem* specificati sotto:

```
//@ requires true ;
//@ ensures (*restituisce l'ordine della matrice quadrata*) ;
public int /*@pure@*/ lunghezza() ;

//@ requires i >= 0 && i < lunghezza() && j >=0 && j<lunghezza() ;
//@ ensures (*restituisce l'elemento in posizione i,j dove i rappresenta il numero della riga e j il numero della colonna*)
public int /*@pure@*/ elem(int i, int j) ;
```

a- Si scriva la specifica formale in JML del metodo leggiMatrice ricordando che la postcondizione deve definire il corretto ordine degli elementi contenuti nel vettore di ritorno.

```
//@ requires true;

//@ ensures
/result !=null &&
\result.length = lunghezza()*lunghezza() &&
(\forall int i; i>=0 && i < lunghezza() ;
                (\forall int j ; j>=0 && j < lunghezza() ;
                        \result[i* lunghezza()+j] == elem(j, i) )) ;

//@assignable \nothing; --oppure si dice esplicitamente che tutti gli  elem(i,j) non cambiano

public int[] leggiMatrice ()
```

b- Supponendo che la rappresentazione (REP) sia la seguente :

```
private int[][] m;
```

si scriva in Java un opportuno iteratore per leggere gli elementi della matrice, definendo sia il metodo di Matrice che restituisce l'iteratore, sia la classe che definisce l'iteratore (ossia il generatore).
L'iteratore deve scandire la matrice colonna per colonna e quindi la sequenza in cui l'iteratore restituisce gli elementi deve ancora essere: primo elemento della prima colonna, secondo elemento della prima colonna, ultimo elemento della prima colonna, primo elemento della seconda colonna, …, ultimo elemento dell'ultima colonna.

Metodo iteratore :

```
public  Iterator<Integer> getIterator() {

        return new MatriceGen(this) ;
}
```

dentro la classe Matrice:

```
private  static class MatriceGen implements Iterator<Integer>{
        private int riga;
        private int colonna;
        private Matrice mat;

        public MatriceGen(Matrice x) {
        mat=x ; riga=0 ; colonna=0 ;
        }
        public boolean hasNext() {
                return colonna<mat.lunghezza();
      }

      public Integer next() throws NoSuchElementException {
                if (colonna>=mat.lunghezza()) throw new NoSuchElementException();
                Integer ret = new Integer(mat.m[riga][colonna]);
                riga++; //si posiziona sulla riga successiva
                if (riga == mat.m.length) {//se le righe sono finite
                  riga = 0; colonna++; //passa alla prima riga della colonna successiva
                }
                return ret;
      }
}
```

NB : Per l'iteratore c'e' anche una soluzione molto semplice, anche se poco efficiente:
si costruisce un array con la leggiMatrice(), se ne ricava una lista (tramite ad esempio il metodo Arrays.asList(), descritto dalla documentazione Java) e poi si ritorna l'iteratore standard di questa lista.
Basta quindi definire il metodo iterator comse segue, senza definire una classe interna:

```
public  Iterator<Integer> getIterator() {
        return Arrays.asList(this.leggiMatrice()).iterator();
}
```

L'iteratore quindi si sposta su una lista che è un nuovo oggetto che contiene i valori della matrice orginale.

Esercizio 2 (punti 12)

Si consideri una versione semplificata del Sudoku, noto gioco solitario di logica. La classe **Sudoku** rappresenta la griglia del gioco. Si ricorda che la griglia è una matrice quadrata di 9x9 caselle. Ciascuna casella può essere vuota oppure contenere un numero fra 1 e 9. Inizialmente la griglia contiene da 20 a 35 numeri e il giocatore deve riempire le caselle vuote disponendo i numeri ---sempre compresi tra 1 e 9--- in modo che ogni riga e ogni colonna non contenga numeri ripetuti. Le righe e le colonne sono numerate da 0 a 8, da sinistra a destra e dall'alto verso il basso. NB: In questa versione semplificata, il piano NON viene diviso in regioni rettangolari in cui controllare l'unicità dei numeri, come invece avviene nel Sudoku "tradizionale".
L'utente avrà a disposizione un'opportuna interfaccia per inserire i numeri e visualizzare la griglia, ma a noi interessa concentrarci sulla logica del gioco e quindi la classe **Sudoku** offre i metodi:

- *riga* e *colonna* per ottenere la parte di griglia richiesta come ArrayList di interi. Le celle vuote sono inizializzate a zero.

- *inserisci* per aggiungere un numero n in posizione x,y della griglia. Il metodo scrive il numero n nella posizione indicata anche se questa è già occupata da un altro numero. Se l'inserimento non viola le regole, il metodo ritorna true; altrimenti il numero non viene inserito e il metodo ritorna false.
- *grigliaCompleta* per capire se una griglia è stata completata in modo corretto. Il metodo restituisce true se la griglia è completa e finita; false altrimenti.

a- Scrivere la specifica del metodo *colonna*:

```
//@ requires
 x >=0 && x <= 8;

//@ ensures
 \result.size() == 9 && (\forall int i; 0 <= i < 9; \result.get(i) == riga(i).get(x));

public ArrayList<Integer> /*@pure@*/ colonna(int x)
```

b- Completare la specifica del metodo *inserisci*:

```
//@ requires n>=1 && n<=9 && x>=0 && x<=8 && y>=0 && y<=8;
```

NB: questa versione di inserisci ipotizza che n>=1, e quindi non puo' essere usata per cancellare una casella

```
//@ ensures
(\forall int i; i>=0 && i<=8; /*elementi non in (x, y) non cambiano*/
        (\forall int j; j>=0 && j<=8;
                (i != x || j != y) ==> riga(i).get(j) == \old(riga(i).get(j)))) &&
(\forall int i; i>=0 && i<=8 ;
        (i!=y==> riga(x).get(i) !=n) && /*nessuna ripetizione sulla riga x*/
        (i!=x; colonna(y).get(i) != n))) /*nessuna ripetizione sulla colonna y*/
? \result == true && riga(x).get(y) == n /*se nessuna ripet. risultato e' true*/
: \result == false && riga(x).get(y) == \old(riga(x).get(y))) ; /*altrimenti false*/



public boolean inserisci(int n, int x, int y)
```

c- Scrivere la specifica del metodo grigliaCompleta. Si faccia attenzione al fatto che il metodo *inserisci* non consente l'aggiunta di elementi che violano le regole del gioco.

```
//@ requires true
/* NB: non e' errato supporre come requires che la griglia non violi le regole del gioco, anche se
in realtà sarebbe una condizione da invariante pubblico */
//@ ensures
 \result == true <==>
                (\forall int i; int i>=0 && i<=8;  (\forall int j; j>=0 && j<=8; riga(i).get(j) !=0))
//@assignable \nothing ;
public boolean grigliaCompleta()
```

d- Si scriva l'invariante pubblico della classe **Sudoku** per imporre la presenza di almeno 20 elementi (diversi da zero) nella griglia.

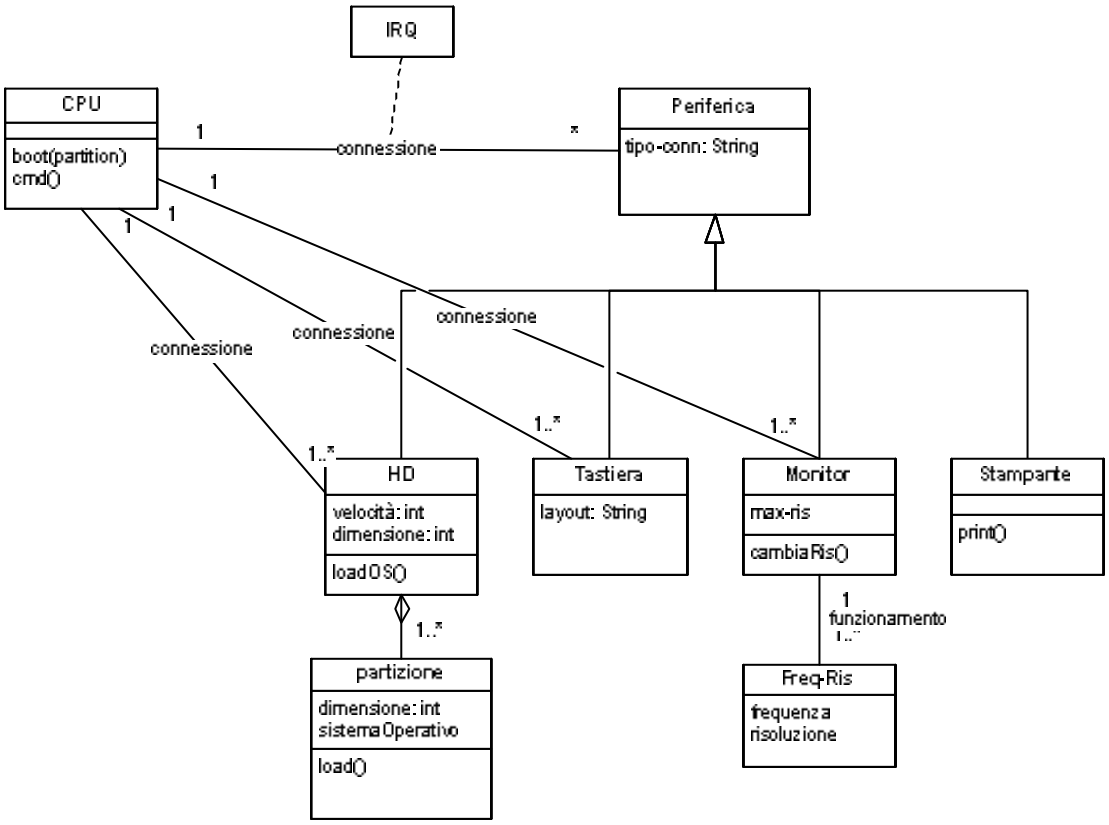
```
//@ public invariant
(\sum_of int i; 0<= i < 9;(\num_of int j; 0 <= j < 9; this.riga(i).get(j) >0)) >= 20;
```

Esercizio 3 (punti 7)

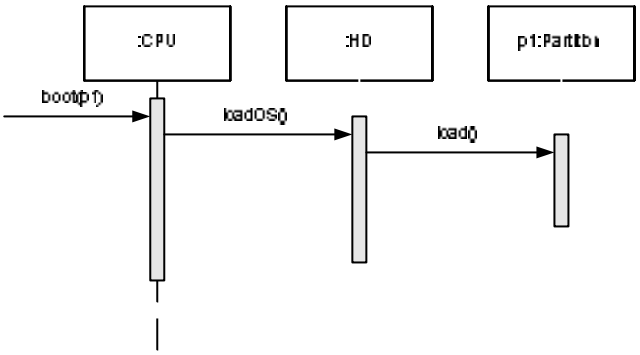
Un computer è composto da una CPU e da alcune periferiche. Le periferiche possono essere tastiere, dischi rigidi, monitor, stampanti (laser o a getto di inchiostro) e altro. Alla CPU devono essere sempre connessi almeno una tastiera, un disco e un monitor.
Di ogni periferica si conosce il tipo di connessione fisica (USB, PS2, ATA, VGA, DVI, …). Le connessioni tra CPU e periferiche sono caratterizzate da un numero di IRQ per la gestione degli interrupt.
La tastiera è caratterizzata da un layout. Del disco rigido sono note velocità di rotazione e dimensione. I dischi sono divisi in partizioni (almeno una) di dimensioni diverse, ognuna formattata con un File System e con un sistema operativo. Ogni monitor ha una risoluzione massima e offre un elenco di coppie risoluzione-frequenza a cui può funzionare.
La CPU reagisce al comando di boot proveniente dall'esterno caricando il sistema operativo dalla partizione selezionata dall'utente. Avviato il sistema, riceve comandi dalla tastiera e li interpreta, agendo eventualmente sulle altre periferiche.

a- Tracciare il class diagram UML descritto sopra.
b- Disegnare il sequence diagram che modella l'accensione del sistema
c- Disegnare il sequence diagram dell'interazione di un utente che – tramite tastiera – invoca comandi sulla CPU. Se il comando è "cambia-risoluzione", la CPU modifica la risoluzione di un monitor in base ai parametri provenienti sempre dalla tastiera.

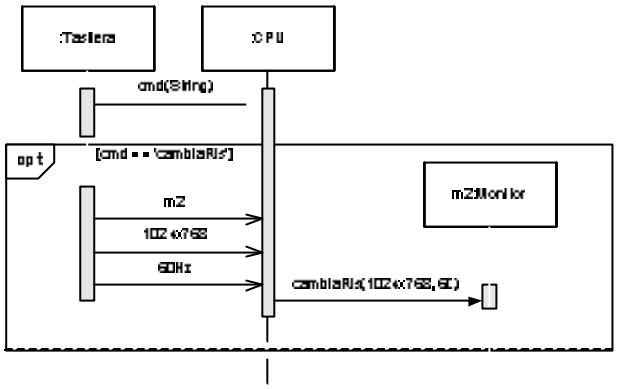
SOLUZIONI UML
a)



b)


:CPU
:HD
p1:Partição
boot(p1)
loadOS()
load()

c)


:Teclado
:CPU
cmd(String)
opt
[cmd == 'cambiaRis']
m2:Monitor
m2
1024x768
60Hz
cambiaRis(1024x768, 60)